

Supporting Evidence – Developer Scoring Format Comparison

StackEval Benchmark Project | Comparison of presentation formats, pilot feedback, and final selected direction

Purpose	Method used	Outcome
To compare different answer-presentation formats for fast and reliable developer scoring.	Visual comparison of three formats plus one pilot test with timing and usability feedback.	A refined final interface that combines readable formatting with direct scoring and automatic result capture.

Document purpose

This supporting evidence document compares the main scoring-format versions that were created for the StackEval benchmark project. The goal was to find a structure that developers could read quickly, score with low confusion, and use without extra manual administration. The final direction was not chosen only by personal preference. It was refined after a small pilot test and feedback session with a developer tester.

Note. The full interview transcript is kept as a separate evidence artifact and is not repeated in full inside this document. This file focuses on the visual comparison, the practical observations, and the final design decision.

Version 1 – Plain text export

```
Question:
Question:
Why am I getting a segmentation fault when accessing an array in C?

Code:
#include <stdio.h>

int main() {
    int *arr;
    arr[0] = 10;
    printf("%d\n", arr[0]);
    return 0;
}

===== Frontier proprietary model =====
Model: gpt-5.4

`arr` is a pointer, but it does not point to valid memory.

### What is wrong
...c
int *arr;
arr[0] = 10;
...

Here, `arr` is an uninitialized pointer. It contains a garbage address, so writing to `arr[0]` tries to access memory you do not own. That causes a segmentation fault.

### Correct fix
Allocate valid memory for the pointer before using it.

### Corrected code
...c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr = malloc(sizeof(int));
    if (arr == NULL) {
        return 1;
    }

    arr[0] = 10;
    printf("%d\n", arr[0]);

    free(arr);
    return 0;
}
...

### Summary
```

Figure 1. Plain text export used as the earliest scoring format.

Observation. This version presents the question, code, model name, and answer as one long text block. It is technically complete, but visually dense.

Main strength. Easy to generate and useful as a raw output record.

Main limitation. Readability is weak. Questions, code, answers, and model identities are mixed together, which increases scanning effort and makes scoring slower. In the pilot feedback, this format was seen as the least readable because it lacked clear structure.

Version 2 – Structured readable layout

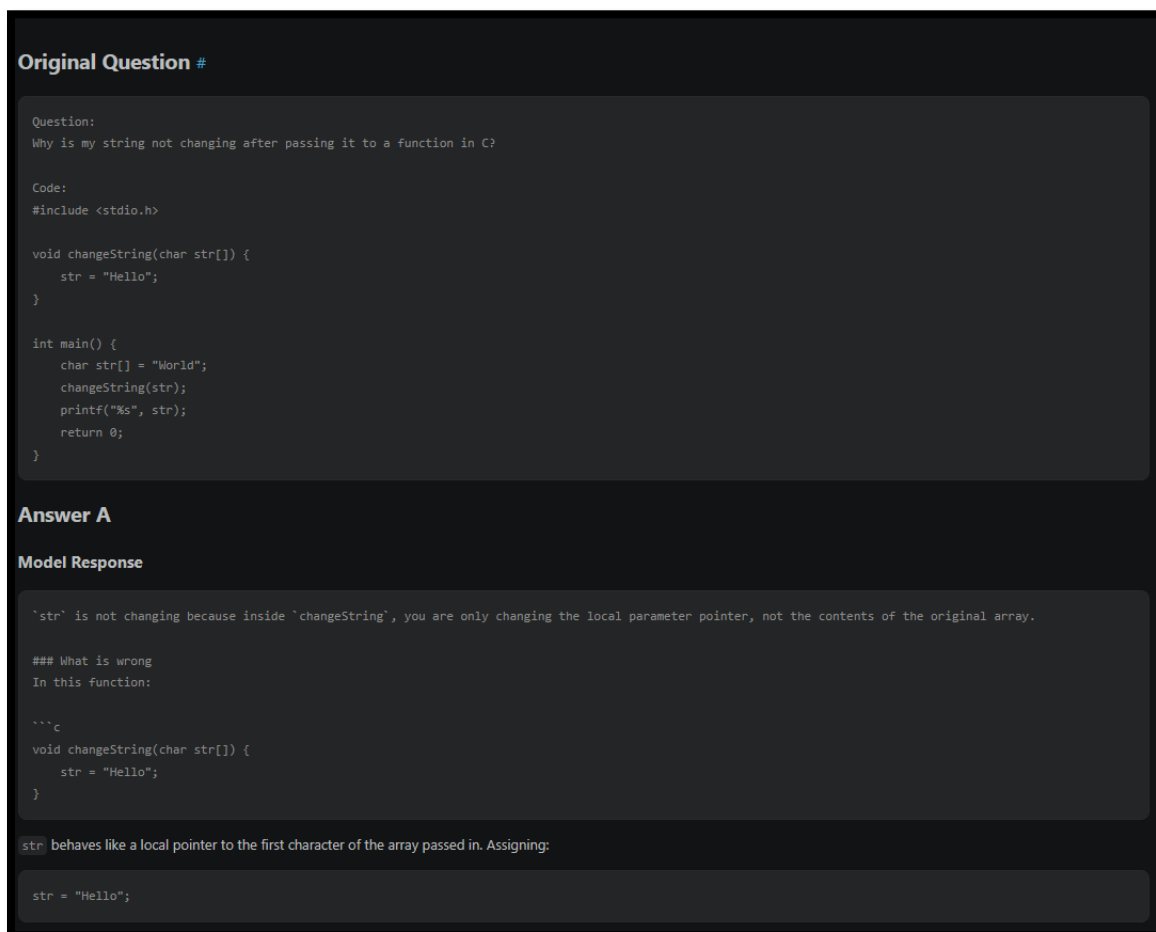


Figure 2. A more structured format with separated content blocks and clearer hierarchy.

Observation. This version improves clarity by separating the original question and the model answer into distinct visual sections. Code blocks are shown more clearly, and the layout is easier to follow than the plain text export. In addition, the model identities are hidden, which supports a more objective scoring process by reducing the chance that developers are influenced by brand or model familiarity.

Main strength. Much better readability and more neutral presentation. The question is easier to understand first, the answer becomes easier to inspect afterward, and hidden model names help developers focus on answer quality rather than the source model.

Main limitation. Although the presentation is stronger, this version still functions mainly as a reading format. It does not yet provide a direct scoring workflow or automatic result collection.

Version 3 – Interactive scoring interface

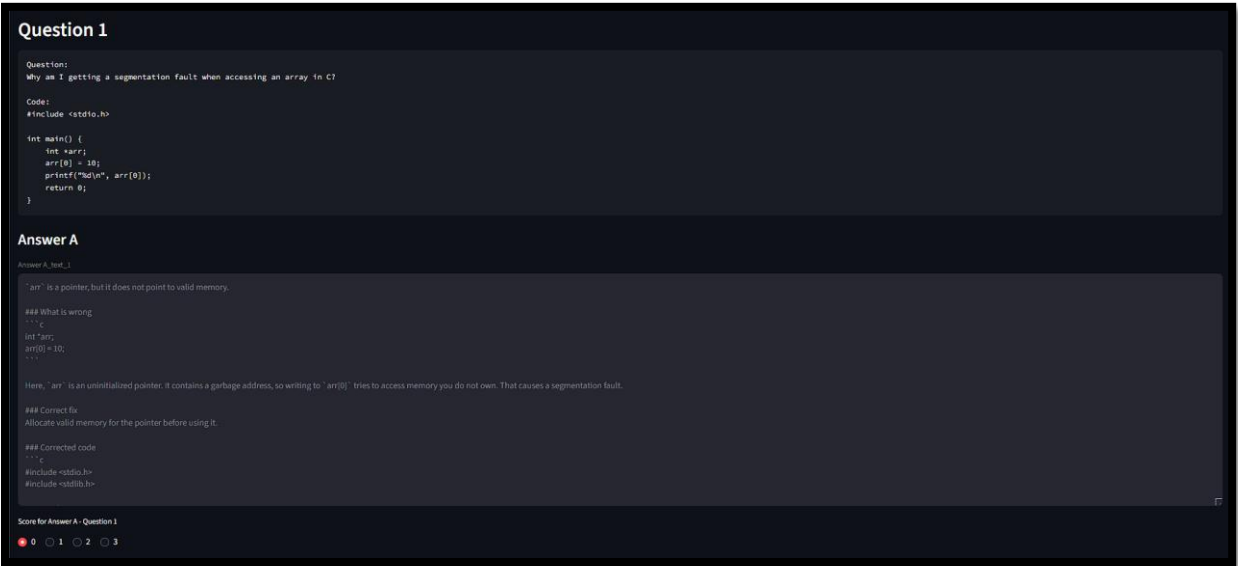


Figure 3. Interactive scoring interface where the developer can score each answer while reading it.



Figure 4. Two practical evidence snapshots from the pilot: saved scoring output (left) and timing measurement during the test session (right).

Observation. This version turns the reading format into an evaluation workflow. The developer can read one answer at a time, assign a score immediately, and submit the results to an Excel file for later analysis.

Main strength. This was the most usable version during the pilot. It supports direct scoring, clearer workflow control, and automatic storage of the results.

Main limitation. The early interface still displayed some answers as large continuous blocks. Based on feedback, the separation between explanatory text and code still needed improvement so the reading experience would feel cleaner and faster.

Final result after feedback

Answer C

Step-by-step analysis of the problem:

1. The code declares a pointer `arr` to an integer. The line `int *arr;` declares a pointer `arr` that is supposed to point to an integer. However, it does not allocate any memory for the integer.
2. The pointer is not initialized: The pointer `arr` is not initialized to point to a valid memory location. It contains an indeterminate value, which is often referred to as a "garbage value."
3. Accessing the pointer as an array: The line `arr[0] = 10;` attempts to access the memory location pointed to by `arr` as if it were an array. Since `arr` is not pointing to a valid memory location, this results in **undefined behavior**.
4. The undefined behavior manifests as a segmentation fault: In this specific case, the undefined behavior results in a segmentation fault because the program is trying to access a memory location that it is not allowed to access.

Fixed solution:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Allocate memory for one integer
    int *arr = malloc(sizeof(int));

    if (arr == NULL) {
        // Handle memory allocation failure
        printf("Memory allocation failed\n");
        return 1;
    }

    arr[0] = 10;
    printf("Value: %d\n", arr[0]);

    // Don't forget to free the allocated memory
    free(arr);

    return 0;
}
```

Explanation of changes:

Figure 5. Refined final direction after feedback: clearer separation between explanation and code inside the scoring interface.

Final direction. The selected design combines the strongest parts of Version 2 and Version 3. It keeps the readable content hierarchy from the structured layout and adds the practical scoring workflow from the interactive interface.

What changed after feedback. The final direction improves the internal structure of each answer. Explanatory text and code are shown more clearly as separate elements, which reduces visual overload and makes the answer easier to inspect during scoring.

Why this version was selected. The pilot feedback indicated that the scoring interface was the best overall format, but it became stronger once the readability improvements were added. This made the final version both usable and visually clearer for developer evaluation.

Comparison summary

Version	Main strength	Main limitation	Contribution to final result
Version 1	Simple raw export	Low readability and visible model labels	Helped show why a more structured format was needed
Version 2	Clearer visual hierarchy and better code separation, and hidden model identities	No direct scoring or automatic saving	Contributed the readable presentation layer
Version 3	Direct scoring and saved results in one workflow, and hidden model identities	Needed cleaner separation inside long answers	Contributed the interactive evaluation flow
Final result	Combines readability, hidden model identities, direct scoring, and saved results in one interface	Still depends on human judgment and rubric limits	Combined the strongest practical features of the previous versions

Key findings from the pilot feedback

- The plain text version was judged as the weakest format because it felt dense and difficult to follow.
- The more structured readable layout was seen as a clear improvement because code and content were easier to distinguish.
- The interactive scoring interface was preferred overall because it allowed scoring during reading instead of after reading.
- A practical improvement point was identified: explanation text and code should be separated more clearly inside the answer area.
- The pilot also gave a first timing indication: scoring two questions took about ten minutes in practice, including discussion during the test.

Evidence used for this comparison

- Screenshot set of the three compared formats and the refined final version.
- Pilot test session with a developer tester (Selim Yildirim).
- Structured interview summary and transcript stored separately as
- Timing snapshot used to estimate scoring duration during the pilot.
- Excel export screenshot showing that submitted scores were stored for later analysis.

This document is intended as supporting evidence for the methodology behind the final scoring interface choice in the StackEval benchmark project.